Лабораторная работа №2

Первоначальная настройка git.

Лопатченко Полина Андреевна

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**

4.1 Установка программного обеспечения 8
4.2 Базовая настройка git 8
4.3 Создание SSH ключа 9
4.4 Создание ключа PGP 10
4.5 Настройка GitHub 11
4.6 Добавление GPG ключа в GitHub 12
4.7 Настройка автоматических подписей коммитов git 13
4.8 Настройка gh 13
4.9 Создание репозитория курса на основе шаблона 13
4.10 Настройка каталога курса 14

5 Контрольные вопросы **15**

6 Выводы **18**

Список иллюстраций

4.1 Установка git и gh 8
4.2 Указание имени и email'а владельца репозитория, указание имени начальной ветки и настройка autocrlf и safecrlf 9
4.3 Создание SSH ключа по алгоритму rsa с ключём размером 4096 бит 9
4.4 Создание SSH ключа по алгоритму по алгоритму ed25519 10
4.5 Создание ключа PGP 11
4.6 Демонстрация учётной записи GitHub 12
4.7 Вывод списка ключей 12
4.8 Сохранённый GPG ключ на GitHub 12
4.9 Настроил автоматические подписи git 13
4.10 Процесс авторизации 13
4.11 Создание репозитория курса на основе шаблона 13
4.12 Настройка каталога курса 14

Список таблиц

1 Цель работы

Изучить идеологию и применение средств контроля версий. Освоить умения по работе с git.

2 Задание

- Создать базовую конфигурацию для работы с git.
- Создать ключ SSH.
- Создать ключ PGP.
- Настроить подписи git.
- Зарегистрироваться на Github.
- Создать локальный каталог для выполнения заданий по предмету.

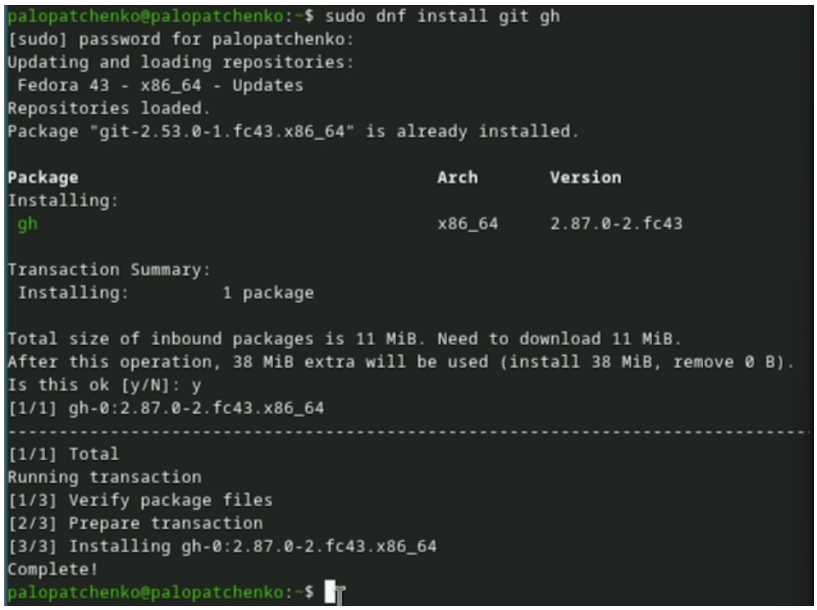
3 Теоретическое введение

Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется. Изучим базовые знания github.

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Установила git и gh (см. рис.1).



```
palopatchenko@palopatchenko:~$ sudo dnf install git gh
[sudo] password for palopatchenko:
Updating and loading repositories:
 Fedora 43 - x86_64 - Updates
Repositories loaded.
Package "git-2.53.0-1.fc43.x86_64" is already installed.

Package                                    Arch       Version
Installing:
 gh                                        x86_64     2.87.0-2.fc43

Transaction Summary:
 Installing:         1 package

Total size of inbound packages is 11 MiB. Need to download 11 MiB.
After this operation, 38 MiB extra will be used (install 38 MiB, remove 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.87.0-2.fc43.x86_64
--------------------------------------------------------------------------------
[1/1] Total
Running transaction
[1/3] Verify package files
[2/3] Prepare transaction
[3/3] Installing gh-0:2.87.0-2.fc43.x86_64
Complete!
palopatchenko@palopatchenko:~$
```

Рисунок 4.1: Установка git и gh

4.2 Базовая настройка git

Задала имя и email владельца репозитория, задала имя начальной ветки, настроила autocrlf и safecrlf (см. рис.2).

```
palopatchenko@palopatchenko:~$ git config --global user.name "Polina Lopatchenko"
palopatchenko@palopatchenko:~$ git config --global user.email "lopatapol228@gmail.com"
palopatchenko@palopatchenko:~$ git config --global core.quotepath false
palopatchenko@palopatchenko:~$ git config --global init.defaultBranch master
palopatchenko@palopatchenko:~$ git config --global core.autocrlf input
palopatchenko@palopatchenko:~$ git config --global core.safecrlf warn
```

Рисунок 4.2: Указание имени и email'а владельца репозитория, указание имени начальной ветки и настройка autocrlf и safecrlf

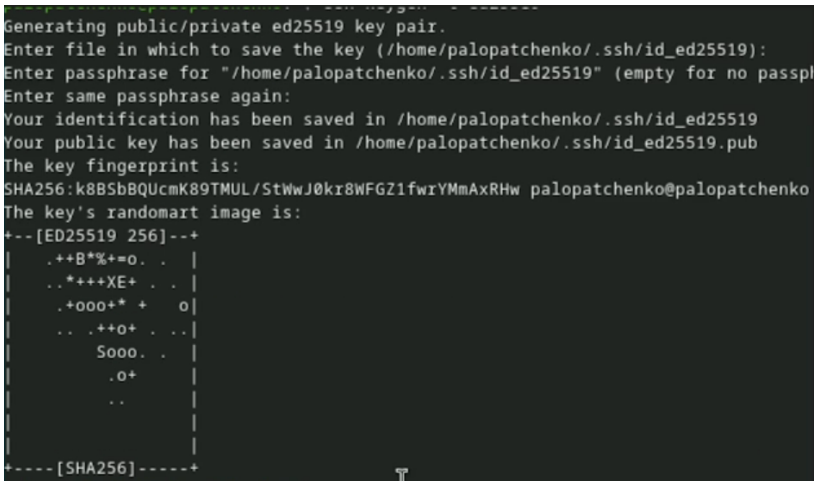
4.3 Создание SSH ключа

- по алгоритму rsa с ключём размером 4096 бит (см. рис.3).

```
palopatchenko@palopatchenko:~$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/palopatchenko/.ssh/id_rsa):
Created directory '/home/palopatchenko/.ssh'.
Enter passphrase for "/home/palopatchenko/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/palopatchenko/.ssh/id_rsa
Your public key has been saved in /home/palopatchenko/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:eqHtT9i9w3ogffPbruKGrR5YQ0gioMvSwTI3RdClXjY palopatchenko@palopatchenko
The key's randomart image is:
+---[RSA 4096]----+
|  o=+.o .        |
| o ..o o .       |
|+ = . E . .      |
|.* + o . .       |
|o.. .   S.o      |
|.      +.*ooo    |
|      o =.+*.o   |
|       o ...B... |
|         .o=*ooo+o|
+----[SHA256]-----+
```

Рисунок 4.3: Создание SSH ключа по алгоритму rsa с ключём размером 4096 бит

- по алгоритму ed25519 (см. рис.4).



```
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/palopatchenko/.ssh/id_ed25519):
Enter passphrase for "/home/palopatchenko/.ssh/id_ed25519" (empty for no passpl
Enter same passphrase again:
Your identification has been saved in /home/palopatchenko/.ssh/id_ed25519
Your public key has been saved in /home/palopatchenko/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:k8BSbBQUcmK89TMUL/StWwJ0kr8WFGZ1fwrYMmAxRHw palopatchenko@palopatchenko
The key's randomart image is:
+--[ED25519 256]--+
|   .++B*%+=o. .  |
|   ..*+++XE+ . . |
|    .+ooo+* +   o|
|    .. .++o+ . ..|
|        Sooo. .  |
|         .o+     |
|         ..      |
|                 |
|                 |
+----[SHA256]-----+
```

Рисунок 4.4: Создание SSH ключа по алгоритму по алгоритму ed25519

4.4 Создание ключа PGP

Создала ключ PGP (см. рис.5).

```
palopatchenko@palopatchenko:~$ gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: directory '/home/palopatchenko/.gnupg' created
Please select what kind of key you want:
   (1) RSA and RSA
   (2) DSA and Elgamal
   (3) DSA (sign only)
   (4) RSA (sign only)
   (9) ECC (sign and encrypt) *default*
  (10) ECC (sign only)
  (14) Existing key from card
Your selection? 1
RSA keys may be between 1024 and 4096 bits long.
What keysize do you want? (3072) 4096
Requested keysize is 4096 bits
Please specify how long the key should be valid.
         0 = key does not expire
      <n>  = key expires in n days
      <n>w = key expires in n weeks
      <n>m = key expires in n months
      <n>y = key expires in n years
Key is valid for? (0) 0
Key does not expire at all
Is this correct? (y/N) y

GnuPG needs to construct a user ID to identify your key.

Real name: Polina Lopatchenko
Email address: lopatapol228@gmail.com
```

Рисунок 4.5: Создание ключа PGP

4.5 Настройка GitHub

В моём случае учётная запись на GitHub с основными данными уже была создана (см. рис.6).

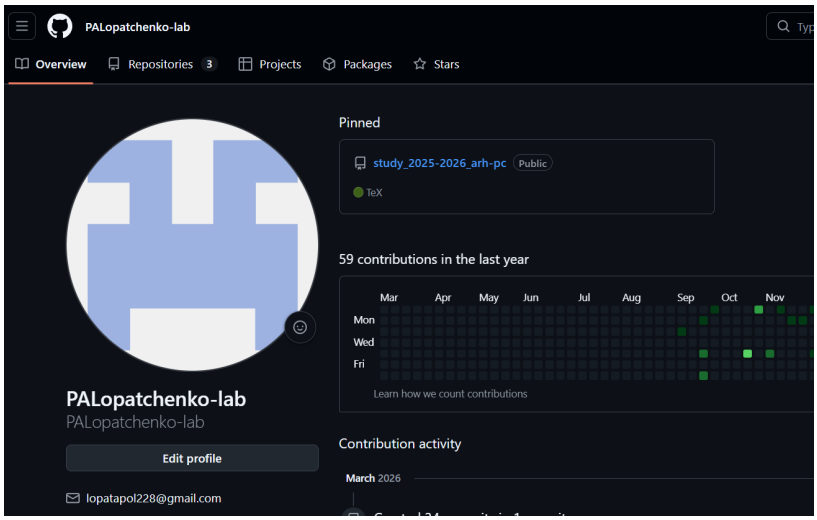


Рисунок 4.6: Демонстрация учётной записи GitHub

4.6 Добавление GPG ключа в GitHub

Вывела список ключей (см. рис.7).

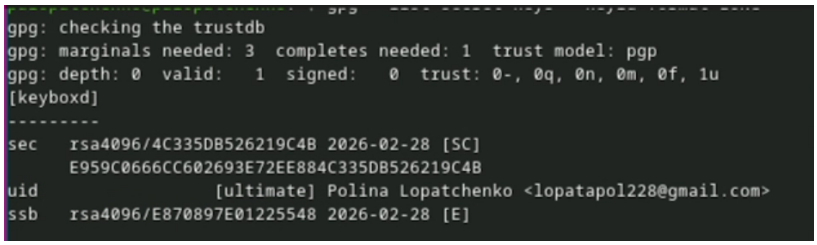


Рисунок 4.7: Вывод списка ключей

Перешла в настройки GitHub, нажала на кнопку New GPG key и вставила полученный ключ в поле ввода (см. рис.9).

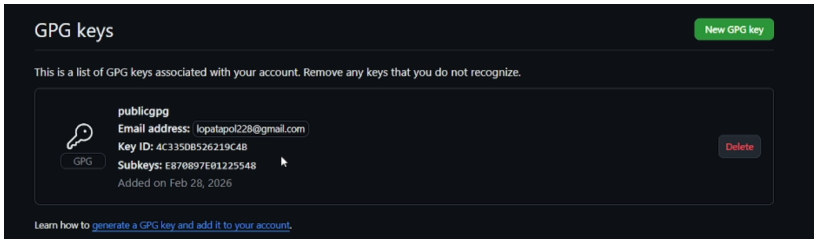


Рисунок 4.8: Сохранённый GPG ключ на GitHub

4.7 Настройка автоматических подписей коммитов git

Используя введённый email, указала Git применять его при подписи коммитов (см. рис.10).

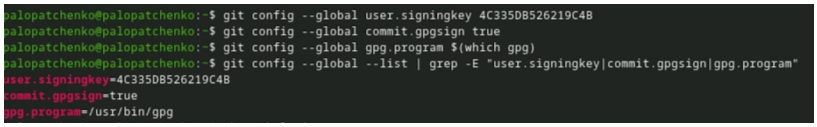


Рисунок 4.9: Настроил автоматические подписи git

4.8 Настройка gh

Авторизация `gh auth login` (см. рис.11).

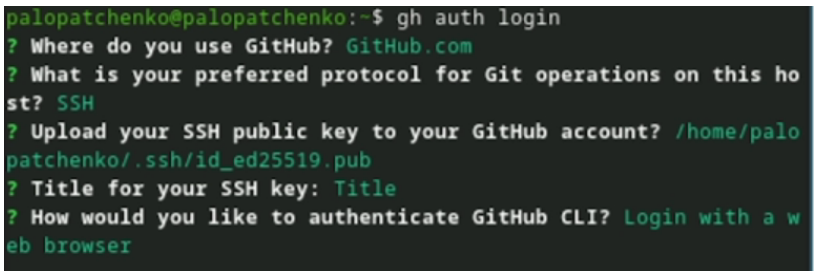


Рисунок 4.10: Процесс авторизации

4.9 Создание репозитория курса на основе шаблона

Создала репозиторий курса на основе шаблона (см. рис.12).

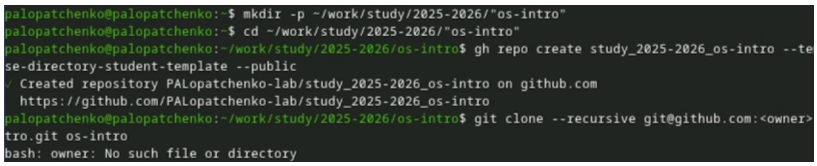


Рисунок 4.11: Создание репозитория курса на основе шаблона

4.10 Настройка каталога курса

Перешла в каталог курса, удалила лишние файлы, создала необходимые каталоги и отправила данные на сервер (см. рис.13).

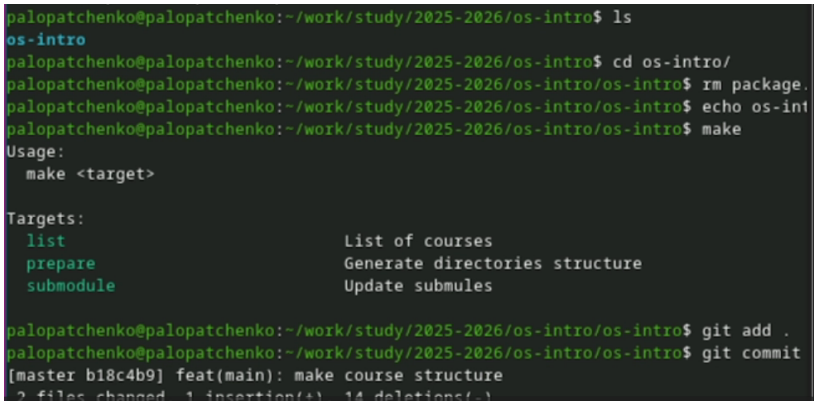


Рисунок 4.12: Настройка каталога курса

5 Контрольные вопросы

1. VCS — инструмент для хранения и управления изменениями файлов проекта. Нужно для ведения истории, отката к прошлым состояниям, совместной работы, контроля кто, когда и что изменил и для безопасного объединения правок.
2. Объяснение понятий VCS:

- Хранилище — место, где хранится проект и его история.
- commit — зафиксированный снимок изменений с сообщением, автором и временем.
- история — цепочка коммитов, показывающая развитие проекта.

3. Централизованные и децентрализованные VCS и их примеры:

- Централизованные: один главный сервер, все работают через него (пример: SVN/Subversion, CVS).
- Децентрализованные: у каждого участника полная копия истории, сервер необязателен (пример: Git, Mercurial, Bazaar).

4. Действия с VCS при единоличной работе: Инициализировать репозиторий (`git init`), редактировать файлы, добавлять в индекс (`git add`), фиксировать (`git commit`), смотреть статус и историю (`git status`, `git log`), при необходимости можно откатываться (`git checkout`, `git reset`).

5. Порядок работы с общим хранилищем VCS: Клонировать проект (`git clone`), перед началом работы забрать обновления (`git pull`), создать

ветку под задачу (опционально), внести изменения, `git add`, `git commit`, отправить на сервер (`git push`), при конфликтах — решить и повторить `push/merge`.

6. Основные задачи, решаемые git: Учёт изменений и история, ветвление и слияние, параллельная разработка, синхронизация с удалёнными репозиториями, контроль конфликтов, быстрый откат к нужной версии, резервное хранение истории.

7. Команды git:

- `git init` — создать локальный репозиторий;
- `git pull` — скачать изменения с удалённого репозитория и слить в текущую папку;
- `git push` — отправить коммиты на удалённый репозиторий;
- `git status` — показать, какие файлы изменены, добавлены, неотслеживаюся;
- `git diff` — показать разницу;
- `git add .` — добавить все изменения в индекс;
- `git add file_names` — добавить конкретные файлы в индекс;
- `git rm file_names` — удалить файлы из репозитория и проиндексировать удаление;
- `git commit -am 'Commit description'` — закоммитить уже отслеживаемые файлы;
- `git commit` — создать коммит;
- `git checkout -b branch_name` — создать новую ветку и перейти на неё;
- `git checkout branch_name` — перейти на существующую ветку;
- `git push origin branch_name` — отправить конкретную ветку на origin;
- `git merge --no-ff branch_name` — слить ветку в текущую, сохранив отдельный merge-коммит;

- `gir branch -d branch_name` — удалить локальную ветку;
- `git branch -D branch_name` — удалить локальную ветку принудительно;
- `git push origin: branch_name` — удалить ветку на удалённом репозитории;

8. Примеры работы с локальным и удалённым репозиторием:

- Локально: `git init`, `git add .`, `git commit -m "msg"`, `git log`.
- Удалённо: `git clone <url>`, `git pull`, `git push`, `git remove -v`.

9. Ветка — отдельная линия разработки в репозитории. Нужна, чтобы делать новую функцию, исправление или эксперемент, не ломая основную ветку, а потом аккуратно слить результат обратно.

10. Ненужные файлы можно добавлять в `.gitignore`, чтобы Git не предлагал их коммитить.

6 Выводы

В ходе данной лабораторной работы я изучил идеологию и применение средств контроля версий и освоил умения по работе с git.